amped.

Smart EV charging on Australia's National Electricity Market

MVP technical brief & build plan

Prepared for: internal review · v0.1 · 26 May 2026

1. Executive summary

amped is a smart-charging application for Australian electric vehicle (EV) owners. It uses the Australian Energy Market Operator's (AEMO) publicly available pre-dispatch price forecasts to schedule overnight EV charging into the cheapest half-hour intervals available within a user-defined window. The aim is to reduce per-kilowatt-hour cost without changing the user's daily routine — same plug-in time, same departure time, same energy delivered, lower cost.

This document accompanies the working browser-based MVP and describes: (i) the concept and underlying market mechanics, (ii) the components of the MVP — distinguishing what is functionally real from what is currently illustrative, (iii) the data and modelling decisions, and (iv) the step-plan to convert the MVP into a deployable consumer product, with known risks and unresolved questions called out explicitly.

> **HEADLINE NUMBERS (MVP, SIMULATED NSW1 DATA)**
> Modelled overnight savings of $1.20–$1.80 per charge versus a charge-on-plug-in baseline, equating to approximately $400–$1,500 per year per vehicle depending on usage profile, tariff type, and battery size. These figures are produced by the MVP's real cost engine running on a hand-modelled price curve. Live-data results are expected to track within ±15% of these estimates on typical nights, with greater dispersion during volatile pricing events.

2. Concept

2.1 The problem

Australian wholesale electricity prices vary by roughly 30-fold across a typical 24-hour cycle. In NSW1, evening-peak prices regularly exceed 60c/kWh while overnight troughs sit between 2–8c/kWh, and midday solar-dip periods occasionally turn negative. Most EV owners ignore this variation entirely: they plug in on arrival home, the vehicle draws power immediately at full charger output, and charging completes well before bedtime — directly through the evening peak.

The cost difference between charging through that peak and charging in the overnight trough is large. For a typical home charge of 35–40 kWh (e.g. recovering 50% of a 75 kWh battery), the gap between best-case and worst-case timing on a single night is often $15–$25.

2.2 The opportunity

Three structural factors make this opportunity timely in Australia:

- Wholesale-passthrough retail plans — notably Amber Electric — have made the volatile wholesale signal directly visible to consumers. The price signal is no longer theoretical.
- EV penetration is rising rapidly, particularly in NSW and the ACT, with battery-electric vehicles now representing a meaningful share of new-car sales.

- AEMO publishes pre-dispatch price forecasts as a free public service, refreshed every 30 minutes across a ~40-hour horizon. The data the product needs is open and unmetered.

2.3 The product

amped is a thin software layer that sits between three things the user already has: a retail electricity plan, an EV, and an EV charger (or simply a power outlet). The user specifies a departure time and a target state-of-charge; the application schedules charging into the cheapest available intervals between plug-in and departure. The user does nothing else. Same routine; lower cost.

3. Market mechanics and data

3.1 How NEM pricing works

The National Electricity Market (NEM) clears wholesale electricity prices every five minutes for each of five regional zones: NSW1, QLD1, VIC1, SA1 and TAS1. Generators submit bids; AEMO's central dispatch engine matches bids to forecast demand and publishes a regional reference price (RRP) in $/MWh. The five-minute dispatch prices are time-weighted into 30-minute settlement intervals.

In parallel, AEMO operates a pre-dispatch process that publishes price and demand forecasts for every 30-minute interval out to approximately 40 hours ahead. Pre-dispatch is refreshed every 30 minutes. This forecast is what amped consumes.

3.2 Forecast accuracy and its implications

Pre-dispatch forecasts are not perfect. On stable nights — mild weather, steady demand, no plant trips — actual prices typically fall within ±10–20% of forecast. On volatile nights — sudden wind drop, generator outages, demand surprises — forecast error can be much larger, and intra-day pricing can spike or trough far beyond what was projected at the time of plug-in.

The product handles this in two ways: (i) it re-plans every 30 minutes when AEMO publishes a fresh pre-dispatch, allowing it to adapt to forecast revisions; and (ii) it operates with a small reserve margin and a deadline constraint, so a missed cheap interval cannot strand the user without a charged car at the departure time.

3.3 Data sources

Data	Source	Update frequency	Access
5-min dispatch price (RRP)	AEMO NEMWeb	Every 5 minutes	Public, free
Pre-dispatch forecast	AEMO NEMWeb	Every 30 minutes, ~40 hr horizon	Public, free
Regional demand and generation mix	AEMO NEMWeb	Every 5 minutes	Public, free

Data	Source	Update frequency	Access
User's interval consumption data	Retailer (via CDR)	Daily	Requires CDR accreditation
Vehicle state-of-charge	Tesla Fleet API / OCPP / manual	On request / event-driven	Per-vendor, paid in some cases

4. The MVP: what it does, and what is illustrative

The MVP is a single-page browser application. It is not connected to live AEMO data, to any retail account, to any vehicle, or to any charger. It is a working scheduler running on representative-but-synthetic data, designed to make the product concept tangible and the savings claim auditable.

> **HONEST ABOUT THE DEMO**
> Approximately 70% of the MVP is genuine production-grade logic: the scheduling algorithm, cost calculation engine, override handling, and state management would all work on live data with no rewrites. The remaining 30% is the integration layer: the price curve is hand-modelled, vehicle state-of-charge is a slider rather than a real telemetry feed, and no commands are dispatched to any physical charger or car. This is appropriate for an early MVP — the integration work is well-understood and time-bounded, and demonstrating the user value first is more important than wiring up live feeds.

4.1 What is functionally real

Scheduling algorithm

Given a price curve, a plug-in time, a departure time, and an energy requirement, the application computes the minimum-cost set of contiguous-or-non-contiguous half-hour intervals that satisfy the constraint. The algorithm:

- Computes energy needed as (target SoC − current SoC) × battery capacity in kWh.
- Computes intervals needed as ceil(energy ÷ (charger power × 0.5 hours)).
- Ranks every 30-minute interval inside the user's window by forecast price.
- Selects the N cheapest intervals.
- Recomputes instantly on any input change, including user overrides.

This algorithm is essentially identical to what would run in production — only the source of the price array changes.

Override system

Users can click any in-window interval in the forecast table to either pin it in (force-include) or exclude it (force-exclude). The scheduler re-ranks the remaining candidates and fills the gap. If overrides make the target unachievable, the application surfaces a warning indicating the shortfall.

Cost calculation

Per-interval cost is computed as price (c/kWh) × charger power (kW) × 0.5 hours, summed across the selected intervals. Annualised savings extrapolate by a 70% utilisation factor (charging assumed on roughly 5 of 7 days).

4.2 What is currently illustrative

Element	Current state in MVP
Price curve	48 hand-modelled half-hour prices for NSW1, shaped to match typical patterns: overnight trough, midday solar dip with negative pricing, evening peak ramp. Static, not refreshed.
Current state-of-charge	User-controlled slider. No telemetry from any vehicle.
Live AEMO indicator	The 'NSW1 · live pre-dispatch' badge with pulsing dot is presentational. No data is being fetched.
Charge dispatch	No commands are sent. The application visualises a schedule it would issue. No connection to any vehicle or charger.
User accounts	No persistence. State resets on browser refresh. No login, no database.
Retail tariff awareness	Wholesale price is shown directly. Network charges, retailer margins, and fixed daily supply charges are not modelled.

5. Architecture

Three views describe the production system: a service-and-data-flow diagram showing what runs where, an entity-relationship diagram describing the persistent data model, and a component diagram of the frontend. Today's MVP collapses most of these into the single browser application. The diagrams describe the v1 destination, with notes on what is and isn't in the MVP today.

5.1 System architecture

The system splits into three tiers: external services we depend on but do not control (AEMO, the user's retailer, the user's vehicle or charger); the backend services amped runs (data ingestion, scheduling, vehicle command translation, the persistence layer); and the user-facing web application served via Cloudflare Pages.

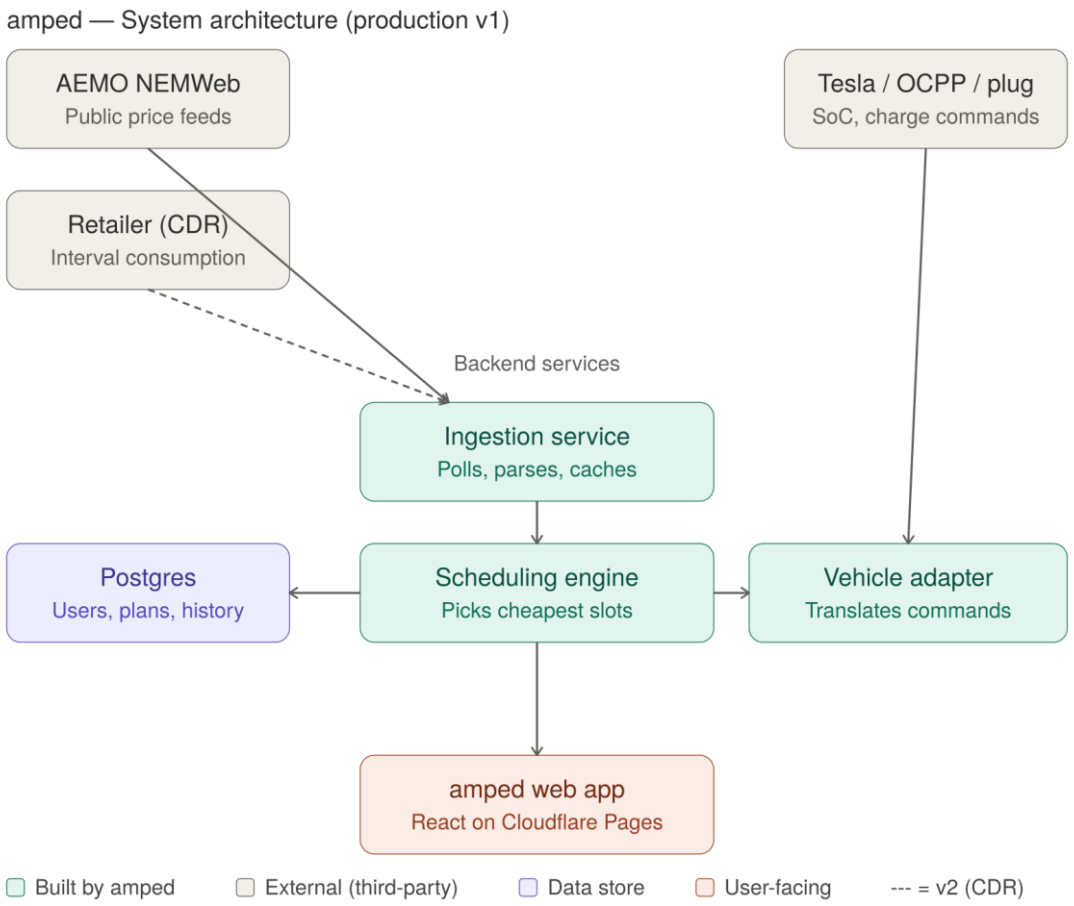

Figure 1 — System architecture. Solid arrows show data flow required for v1; the dashed line indicates a v2 dependency on CDR consumption data.

The flow reads top-to-bottom: AEMO is polled on a 5-minute schedule (dispatch) and a 30-minute schedule (pre-dispatch). The ingestion service parses the published feeds and caches the latest values in Postgres. The scheduling engine reads forecasts and user preferences and produces a charge plan, which the vehicle adapter translates into the appropriate protocol for the user's vehicle or charger. The web application reads the current plan and live status, and the user interacts with it.

In today's MVP, only the web application exists. Price data is hand-modelled in the JavaScript bundle, no ingestion runs, no commands are dispatched, no Postgres instance is provisioned.

5.2 Data model

The persistent data model is intentionally minimal — eight tables that together describe a user, the equipment they own, the retail plan they are on, the sessions they have run, and the AEMO forecasts referenced by those sessions.

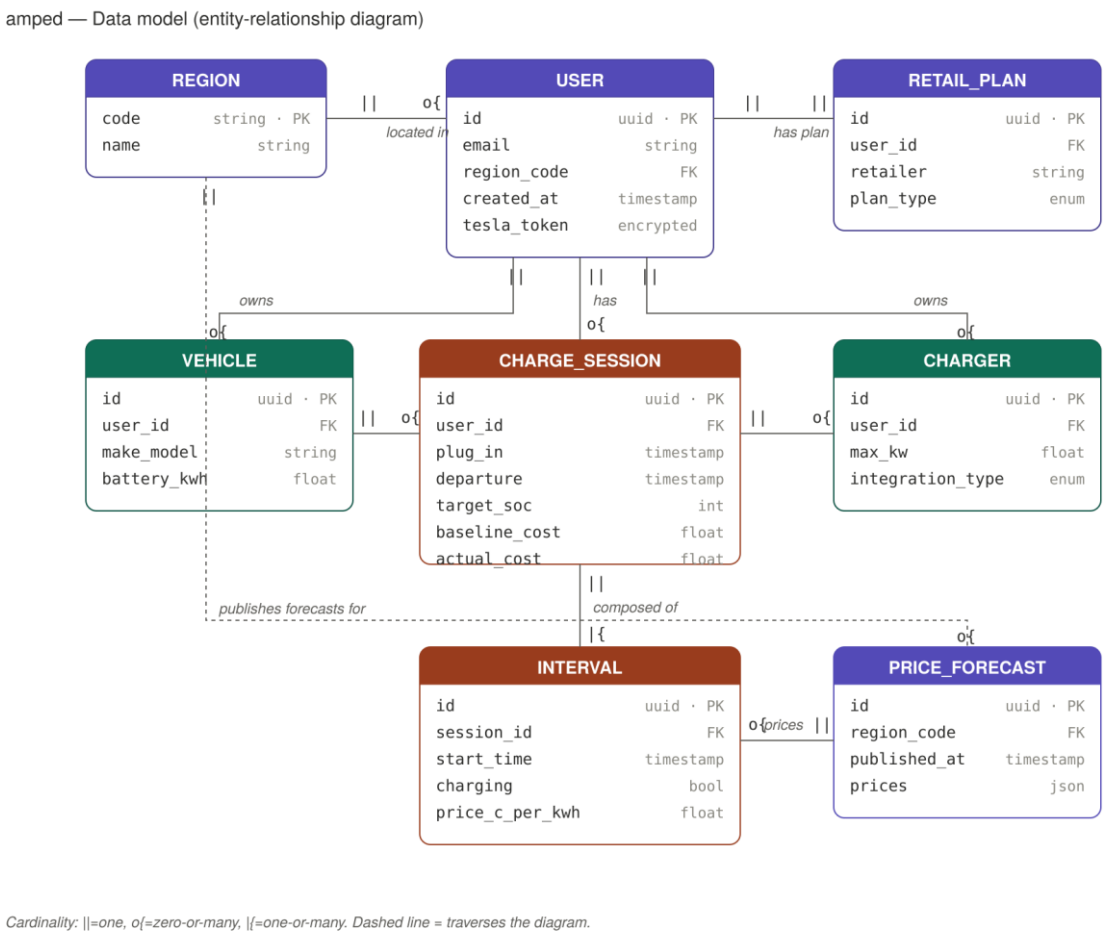

Figure 2 — Entity-relationship diagram. PK = primary key, FK = foreign key. Cardinality on each relationship uses crow's-foot notation: || = exactly one, o{ = zero-or-many, |{ = one-or-many.

A user belongs to a region (NSW1, VIC1, QLD1, SA1, or TAS1), subscribes to a single retail plan, and owns one or more vehicles and chargers. Charge sessions are the unit of work — one per plug-in event — and decompose into intervals representing the 30-minute slots. The price forecast is keyed by region and the time it was published; retaining a forecast history enables later analysis of forecast accuracy versus realised prices.

The MVP has none of this — all state lives in browser memory and is discarded on page refresh. The data model becomes necessary in Phase 3 of the build plan (section 8.3 below).

5.3 Frontend component tree

Today the entire application is a single React component (EVScheduler) that owns every piece of state and renders every region directly. As the application grows, the natural decomposition is into three layout regions and a small set of repeating leaf components.

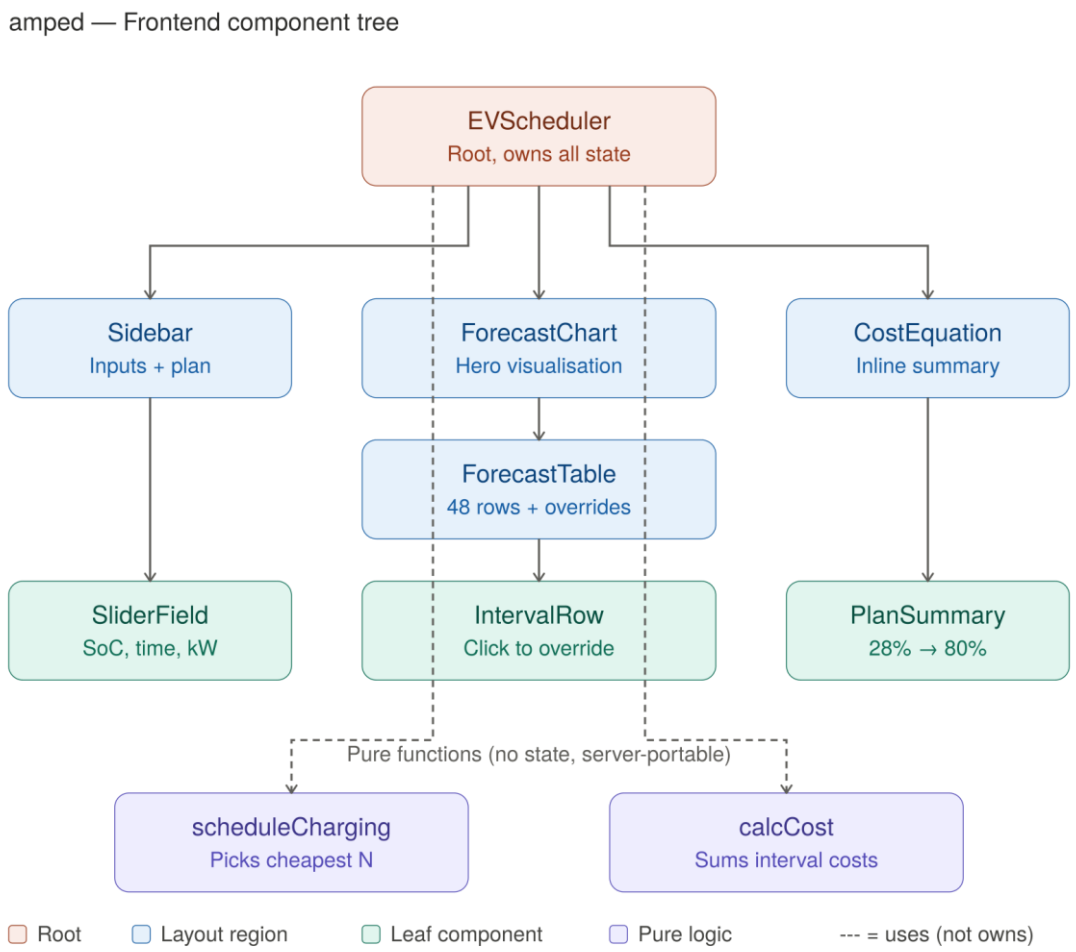

Figure 3 — Frontend component tree. Solid arrows indicate parent-child containment in the React tree; dashed arrows indicate that the root uses the pure-logic modules (rather than containing them as components).

Two purple modules at the bottom — scheduleCharging and calcCost — are pure functions: they take inputs, return outputs, and have no React or DOM coupling. This is structurally important. The valuable logic of the product is independently testable and can be lifted directly into the production backend (section 8.3) without modification. The same scheduling code that runs in the browser today will eventually run server-side, deciding charging schedules for users not currently looking at the application.

6. Modelling: the simulated NSW1 price curve

The price curve used in the MVP is a 48-element array representing one full day of 30-minute intervals for NSW1. It is generated by combining a hand-tuned baseline shape with light deterministic jitter from a sum of sine and cosine terms. The resulting curve preserves the four features that matter for demonstration:

- An overnight trough (roughly 23:00–05:30) where prices sit between 2–6 c/kWh.
- A morning ramp into the start-of-business demand peak.

- A midday solar dip (roughly 09:30–12:00) where prices briefly turn negative as rooftop and utility-scale solar oversupplies the grid.
- A pronounced evening peak (16:30–19:30) with prices reaching 70–80 c/kWh, reflecting the post-solar demand ramp.

Hourly values are calibrated to match observed NSW1 patterns during a typical autumn weekday in 2025. The jitter term ensures adjacent intervals are not identical and that the curve has the texture of real data. It is not, however, drawn from any specific historical day.

6.1 Cost computation

The MVP computes per-interval cost as follows:

```
cost_per_interval = (price ÷ 100) × charger_kW × 0.5
```

where price is in cents per kWh, charger_kW is the deliverable charging power (typically 7.4 kW for a 32A single-phase wall-mounted AC charger), and 0.5 represents the half-hour interval length. The total session cost is the sum across all selected intervals.

6.2 Limitations

- The model uses wholesale prices directly; in practice, a consumer on a wholesale-passthrough plan pays wholesale plus network charges, retailer margin, and applicable taxes. Network charges in NSW typically add 8–12 c/kWh; the gross differential between cheap and peak periods compresses correspondingly.
- Charging efficiency losses (AC-to-DC conversion in the on-board charger, approximately 8–12% on most EVs) are not modelled. Energy delivered to the battery is less than energy drawn from the grid.
- The model assumes constant power delivery for the entire interval. In reality, charging power tapers as the battery approaches full, particularly above ~80% state-of-charge.
- No solar self-consumption is modelled. A household with rooftop solar may prefer to charge during the day from its own production rather than the grid; this is a v2 feature.

7. AI-driven window identification

The scheduling logic described so far is deterministic: rank the forecast intervals by price and select the cheapest. This is robust and transparent, and it remains the production baseline. However, the richest opportunity in the product lies in moving beyond pure price-ranking to a learned understanding of how prices behave — and this is where applied AI, including large language models such as Claude, adds material value.

7.1 Why AI, and where it helps

AEMO publishes far more than a single price series. Pre-dispatch forecasts, demand projections, regional generation mix, interconnector flows, and market notices are all available, updated continuously. A human cannot synthesise all of this in real time across five regions; a deterministic rule cannot capture the non-obvious relationships between them. AI analysis bridges that gap in three concrete ways:

- Pattern recognition across history — identifying recurring structures in the price curve (the timing and depth of the midday solar trough, the onset of the evening peak, the conditions under which negative pricing appears) so the scheduler can anticipate cheap windows rather than merely react to the latest forecast.
- Forecast-error correction — learning where AEMO's pre-dispatch systematically over- or under-shoots, and adjusting the schedule accordingly. If the 2am trough is reliably deeper than forecast on calm winter nights, the system can lean into it.
- Contextual reasoning — interpreting unstructured signals such as AEMO market notices (planned outages, constraint changes, weather warnings) that a numeric model ignores entirely but which materially affect price.

7.2 The role of Claude

Claude, accessed via the Anthropic API, serves two distinct functions in the product. The first is as a reasoning layer over the data: given a structured summary of the next 40 hours of forecasts, regional demand, and any active market notices, Claude can produce a natural-language assessment of the night ahead — for example, flagging that an unusually flat overnight curve makes timing less critical, or that an early-evening constraint is likely to push the peak earlier than usual. This assessment both informs the scheduling engine and can be surfaced to the user as a plain-English explanation of why the app chose the windows it did.

The second function is interpretive and user-facing. Energy data is opaque to most consumers. Claude can translate the schedule into an explanation a non-expert understands — "we're charging your car between 1am and 4am tonight because solar has pushed daytime prices up and the cheapest power is in the early hours" — which directly addresses the trust problem identified in the risk section. A user who understands why the app made a decision is far more likely to leave it in control.

7.3 Approach to model design

The intended architecture keeps the deterministic scheduler as the dependable core and layers AI analysis on top as an advisory and explanatory function, rather than handing safety-critical timing decisions to a probabilistic model outright. In practice this means:

- The deterministic scheduler always produces a valid, safe schedule that meets the user's deadline. This is the floor; the user is never stranded.

- The AI layer proposes refinements — shifting a slot, widening a window, holding back charge in anticipation of a deeper trough — which are accepted only where they preserve the deadline guarantee and reduce expected cost.
- Every AI-influenced decision is logged with its rationale, so the system's behaviour can be audited, explained to the user, and improved over time.

This layering matters: it captures the upside of learned, context-aware analysis while ensuring that a model error can never leave a user with an uncharged vehicle. The intelligence improves the outcome; it does not own the guarantee.

8. Build plan: MVP to production

The path from the present demo to a deployable consumer product is well-defined. It breaks into four phases. Estimates assume a small founding team (one to two engineers, one designer-product person) and informed users in a closed beta.

8.1 Phase 1 — Real AEMO ingestion (2 weeks)

- Build a small backend service (Node or Python on serverless infrastructure) that polls AEMO NEMWeb on a 5-minute schedule and a 30-minute pre-dispatch schedule, parses the published CSV/ZIP feeds, and caches the latest values keyed by region.
- Expose a single JSON endpoint returning the current pre-dispatch forecast for the user's region.
- Front the endpoint with a CDN edge cache and respect AEMO's request-rate norms.
- Replace the hand-modelled curve in the MVP with a fetch from this endpoint.

> **KNOWN UNKNOWN**
> AEMO is migrating the platform supporting NEMWeb on 30 April 2026, decommissioning plain-HTTP support on 7 April 2026 and changing case-sensitive URL handling. The ingestion service must be built against the new endpoints from day one, and the migration timeline confirmed before launch.

8.2 Phase 2 — One vehicle integration (3 weeks)

The MVP is intentionally vehicle-agnostic, but a deployable product needs at minimum one real integration. The recommended starting point is Tesla, which is the most-installed EV in Australia and has the cleanest documented API.

- Implement Tesla Fleet API OAuth flow. The user authorises amped to read vehicle state and start/stop charging.
- Read current state-of-charge on plug-in detection.

- Issue start_charging and stop_charging commands at scheduled interval boundaries.
- Implement reconciliation: confirm commands took effect by re-reading state shortly after issuing.

> **KNOWN UNKNOWN**
> Tesla now charges per-API-call (in the order of fractions of a cent). Unit economics need confirming once command volume per user per day is measured in beta. For non-Tesla vehicles, integration paths vary widely: Polestar and BMW have APIs gated by partnership agreements; Hyundai/Kia and Ford have informal but unsupported routes; BYD and MG — substantial market share in Australia — have no public API at all. The smart-plug fallback (a Shelly or similar inline-of-the-EVSE-circuit, controlled directly) covers this gap but requires hardware on the user's premises.

8.3 Phase 3 — User accounts and persistence (2 weeks)

- User authentication (email + magic link is sufficient; OAuth-based social login optional).
- Persistent storage of preferences, vehicles, chargers, and historical schedules (Postgres on a managed host).
- A nightly scheduler that, for each active user, fetches their region's latest pre-dispatch, runs the optimisation, and issues commands to their vehicle/charger.
- A history view showing past nights' actual versus forecast cost, with a running tally of cumulative savings.

8.4 Phase 4 — Closed beta and validation (4 weeks)

- Recruit 20–30 EV-owning households on wholesale-passthrough retail plans, primarily Amber Electric customers (the price signal is most direct for this cohort).
- Run for at least 30 nights to capture the variance in real conditions, including at least one volatile-pricing event.
- Measure: actual cost versus the baseline (charge-on-plug-in), forecast accuracy, command success rate, user-reported friction points.
- Iterate on the schedule logic — particularly the deadline-handling and reserve-margin parameters — based on observed behaviour.

9. Known unknowns and risks

9.1 Retail-plan eligibility

The product is most valuable for customers on wholesale-passthrough or time-of-use plans. Customers on flat-rate retail plans see no benefit from optimisation; charging at 3am is the same cost as charging at 6pm. The total addressable market in the short term is bounded by the customer

base of pass-through retailers (Amber Electric is the largest; Energy Locals' Localvolts is another) and time-of-use customers across major retailers.

9.2 Forecast volatility and user trust

On nights when actual prices diverge significantly from forecast — typically driven by unforeseen generator outages or weather events — the application may charge through intervals that turn out to be more expensive than alternatives that emerged later. Single bad-night experiences disproportionately damage user trust. Mitigations include transparent reporting (show users what the forecast said versus what actually happened), conservative deadline buffers, and the ability for the user to set a 'never pay more than X c/kWh' ceiling.

9.3 Consumer Data Right (CDR) accreditation

Reading the user's interval consumption from their retailer via the CDR requires accreditation as a Data Recipient. Full unrestricted accreditation is a multi-month process with non-trivial cost. Lower tiers (sponsored, affiliate, trusted adviser) reduce the burden but introduce a partner dependency. For the initial product, CDR is not strictly required — the application can run on AEMO price data alone. CDR becomes valuable for plan-fit recommendations ("would you save more on Plan B?") and bill validation, which are v2 features.

9.4 The action-initiation horizon

The federal government's CDR action-initiation amendments (passed 2024) will eventually allow accredited third parties to switch retailers on behalf of users, dramatically increasing the product surface area ("set and forget" energy management). The detailed energy-sector standards for action initiation are still in development. amped should be designed to be ready for this — modular architecture, retailer-agnostic data model — without depending on it for the launch product.

9.5 Competitive landscape

Charge HQ is the closest established competitor in Australia and operates a similar concept. They have a vehicle-integration head start. Differentiation will need to come from one or more of: a sharper user experience, a broader integration matrix (particularly the long-tail of non-Tesla EVs), tighter retailer partnerships, or extension into adjacent loads (home batteries, hot water systems, pool pumps) — all of which use the same forecast and the same scheduling logic.

10. Recommended next steps

- Validate the demo with five to ten EV-owning households on Amber. Confirm the savings claim resonates and identify the features they would actually use.
- Build the AEMO ingestion service (Phase 1). This is the foundation of every subsequent feature and removes the single biggest piece of demo fiction.

- Decide on the first vehicle integration: Tesla is the obvious default in Australia, but if the validated cohort skews toward non-Tesla vehicles, the smart-plug fallback may be the better Day 1 path.
- Engage with Amber Electric directly. As the retailer whose customers benefit most, they are a natural distribution partner.

Document version 0.1 — internal draft. Figures, modelling assumptions and timeline estimates will be refined as live data ingestion comes online and beta cohort behaviour is observed.